

Kubernetes

understanding Kubernetes

Brice Ekane

brice.ekane@univ-

[rennes.fr](https://github.com/)

<https://github.com/>

[bekane/tlc.git](https://github.com/bekane/tlc.git)

2025-2026

Cours repris de Pr. Olivier Barais

Kubernetes: What's it do?

Agenda

- What is Kubernetes
- Kubernetes concepts
- Architecture
- Conclusion

Kubernetes VS Docker compose

Docker

- Docker is the container technology that allows you to containerize your applications.
- Docker is the core of using other technologies.

Kubernetes VS Docker compose

Docker Compose

- Docker Compose allows configuring and starting multiple Docker containers.
- Docker Compose is mostly used as a helper when you want to start multiple Docker containers and don't want to start each one separately using docker run
- Docker Compose is used for starting containers on the same host.
- Docker Compose is used instead of all optional parameters when building and running a single docker container.
- Docker Swarm

Kubernetes VS Docker compose

Kubernetes

- Kubernetes is a container orchestration tool developed by Google
- Kubernetes' goal is very similar to that for Docker Swarm

What is Kubernetes?

Definition: a container orchestrator

- Kubernetes is a container orchestrator like Docker Swarm, Mesos Marathon, Amazon ECS, Hashicorp Nomad. Container orchestrators are the tools which group hosts together to form a cluster, and help us make sure applications:
 - are fault-tolerant,
 - can scale, and do this on-demand
 - use resources optimally
 - can discover other applications automatically, and communicate with each other
 - are accessible from the external world
 - can update/rollback without any downtime.

What is Kubernetes?

Origins and purpose

- A highly collaborative open source project originally conceived by Google
- Google has 10+ years experience w/ containerized apps
- Sometimes called:
 - kube
 - k8s (that's 'k' + 8 letters + 's')
- Start, stop, update, and manage a cluster of machines running containers in a consistent and maintainable way.

What is Kubernetes?

Best fit and scope

- Particularly suited for horizontally scalable, stateless, or 'microservices' application architectures.
 - Does not mean others will not work or are ignored
- Additional functionality to make containers easier to use in a cluster (reachability and discovery).
- **Kubernetes does NOT and will not expose all of the 'features' of the docker command line.**

What is Kubernetes?

Why it feels complex

- Many people argue that Kubernetes is hard to learn.
- It's because it solves a series of problems and people try to understand without knowing all the prerequisites.
- This makes it complicated.
- **Start putting the pieces of the puzzle together by reading about concepts/terms like the following.**
- This process will help you understand the kind of problems Kubernetes tries to solve:

What is Kubernetes?

Key problems Kubernetes solves

- This process will help you understand the kind of problems Kubernetes tries to solve:
 - **12-factor apps**,
 - Automatic binpacking,
 - Self-healing mechanisms,
 - Horizontal scaling,
 - Service discovery and Load balancing,
 - Automated rollouts and rollbacks,
 - Blue-Green deployments / Canary deployments
 - Secrets and configuration management,
 - Storage orchestration

- ▶ And because there are a lot of different things around containers and their management, keep an eye on the Cloud Native landscape: <https://landscape.cncf.io/>

KUBERNETES: CONCEPTS ET OBJETS

Panorama des objets fondamentaux

- Master
- Minion/Node
- Pod
- Replication Controller
- Deployments
- Ingress et Ingress controller
- NetworkPolicy
- Services
- Label
- Namespace

KUBERNETES : Les concepts

KUBERNETES : Les concepts

Le plan de contrôle du cluster

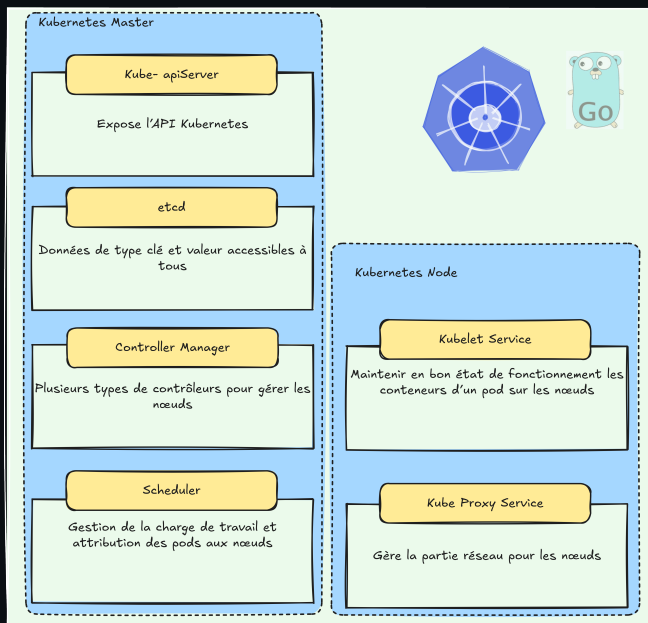
- Master
 - Typically consists of:
 - kube-apiserver
 - kube-scheduler
 - kube-controller-manager
 - etcd
 - Might contain:
 - kube-proxy
 - a network management utility

KUBERNETES : Minion - Node

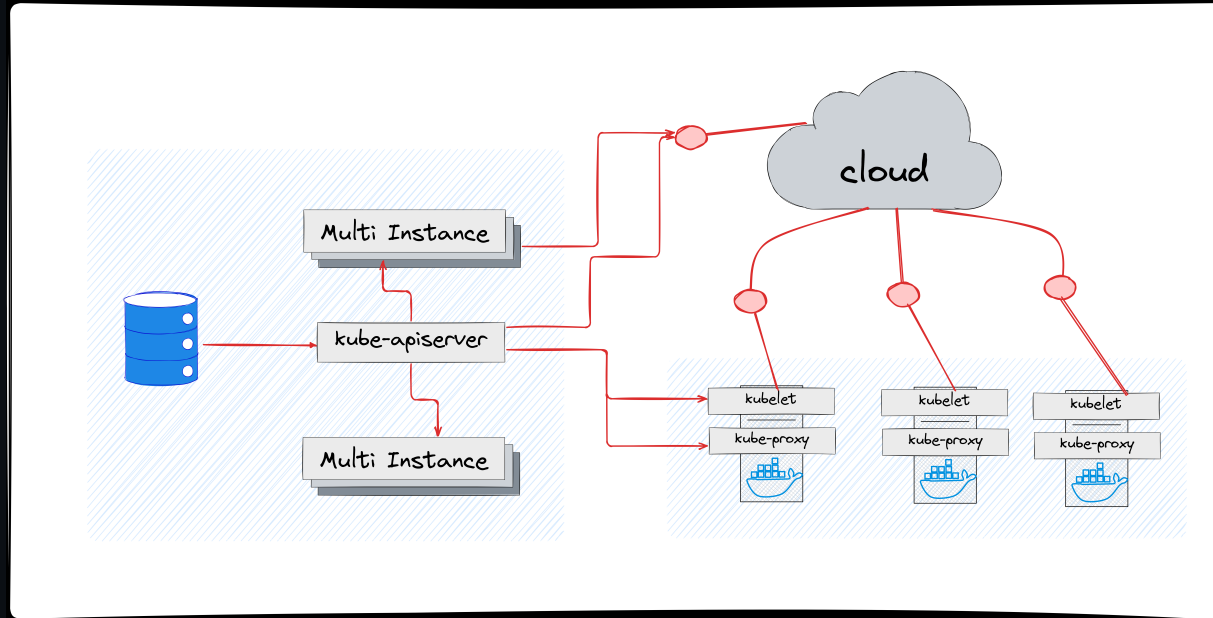
Le nœud de calcul qui exécute les pods

- Typically consists of:
 - kubelet
 - kube-proxy
 - cAdvisor
- Might contain:
 - a network management utility
- May be referred to by either name.

KUBERNETES : Systems and Binaries



KUBERNETES : Systems and Binaries



KUBERNETES : POD

La plus petite unité déployable

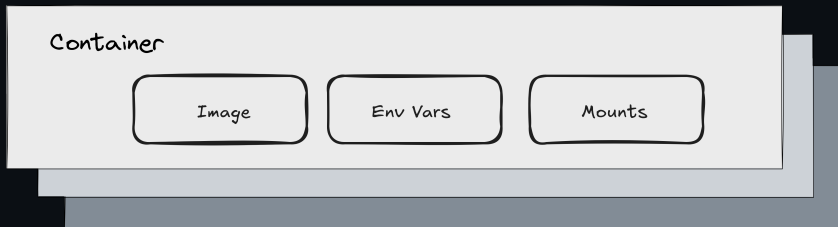
- Ensemble logique composé de un ou plusieurs conteneurs
- Les conteneurs d'un pod fonctionnent ensemble
- (instanciation et destruction) et sont orchestrés sur un même hôte
- Les conteneurs partagent certaines spécifications du POD :
 - La stack IP (network namespace)
 - Inter-process communication (PID namespace)
 - Volumes
- C'est la plus petite unité orchestrable dans Kubernetes

KUBERNETES : POD

Unité de scheduling et d'isolation réseau

- Single schedulable unit of work
- Can not move between machines
- Can not span machines
- One or more containers
- Shared network namespace
- Metadata about the container(s)
- Env vars – configuration for the container
- Every pod gets an unique IP
- Assigned by the container engine, not kube!

POD



Mount Sources

Labels

Annotations

IP Address

KUBERNETES : POD

Déclarer un Pod en YAML

- Les PODs sont définis en YAML comme les fichiers docker-compose
- Exemple minimal :

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

KUBERNETES: Replication Controller

Assurer un nombre constant de Pods

- Consists of
 - Pod template
 - Count
 - Label Selector
- Kube will try to keep \$count copies of pods matching the label selector running
- If too few copies are running the replication controller will start a new pod somewhere in the cluster

KUBERNETES: Replication Controller



KUBERNETES : SERVICES

Exposer des pods via une IP stable

- Abstraction des PODs et RCs, sous forme d'une VIP de service
- Rendre un ensemble de PODs accessibles depuis l'extérieur
- Load Balancing entre les PODs d'un même service

KUBERNETES : SERVICES

Types de Service et exposition réseau

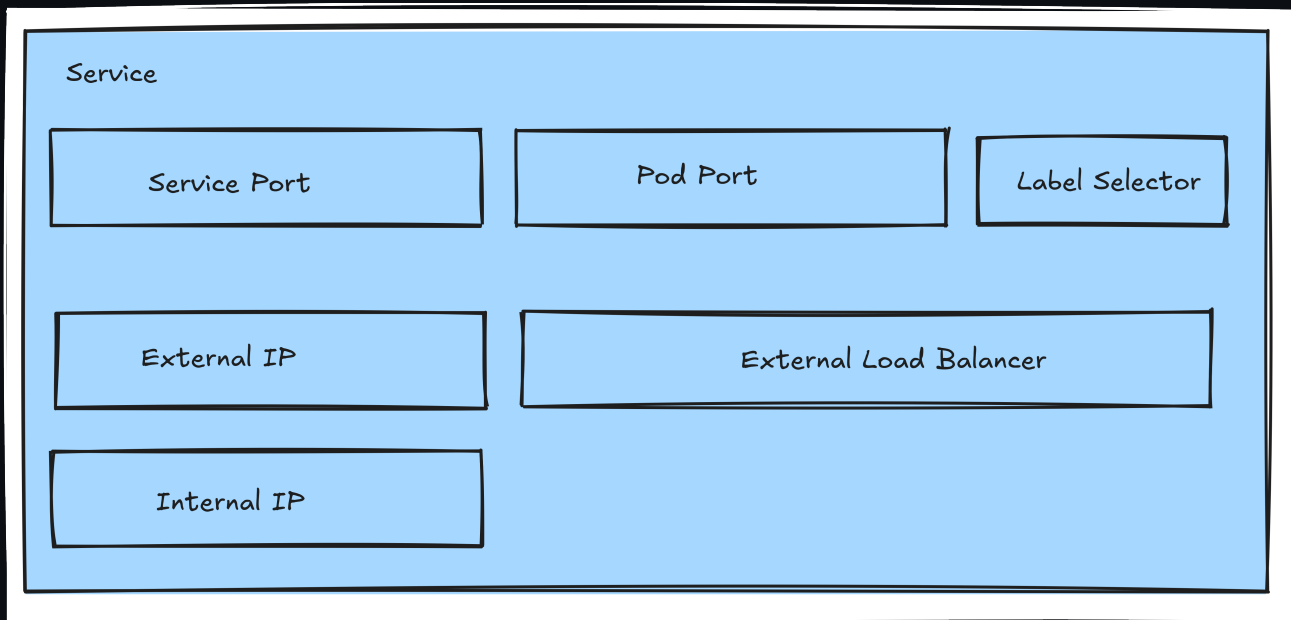
- Load Balancing : intégration avec des cloud provider :
 - AWS ELB
 - GCE
- Node Port forwarding : limitations
- ClusterIP : IP dans le réseau privé Kubernetes (VIP)
- IP Externes : le routage de l'IP publique vers le cluster est manuel

KUBERNETES : SERVICES

Découverte des pods et routage du trafic

- How 'stuff' finds pods which could be anywhere
- Define:
 - What port in the container
 - Labels on pods which should respond to this type of request
- Can define:
 - What the 'internal' IP should be
 - What the 'external' IP should be
 - What port the service should listen on

KUBERNETES : SERVICES



KUBERNETES : SERVICES

Exemple de définition YAML d'un Service

- Exemple de service (on remarque la sélection sur le label):

```
kind: Service
apiVersion: v1
metadata:
  name: example-service
spec:
  ports:
    - port: 8765
      targetPort: 9376
  selector:
    app: example
  type: LoadBalancer
```

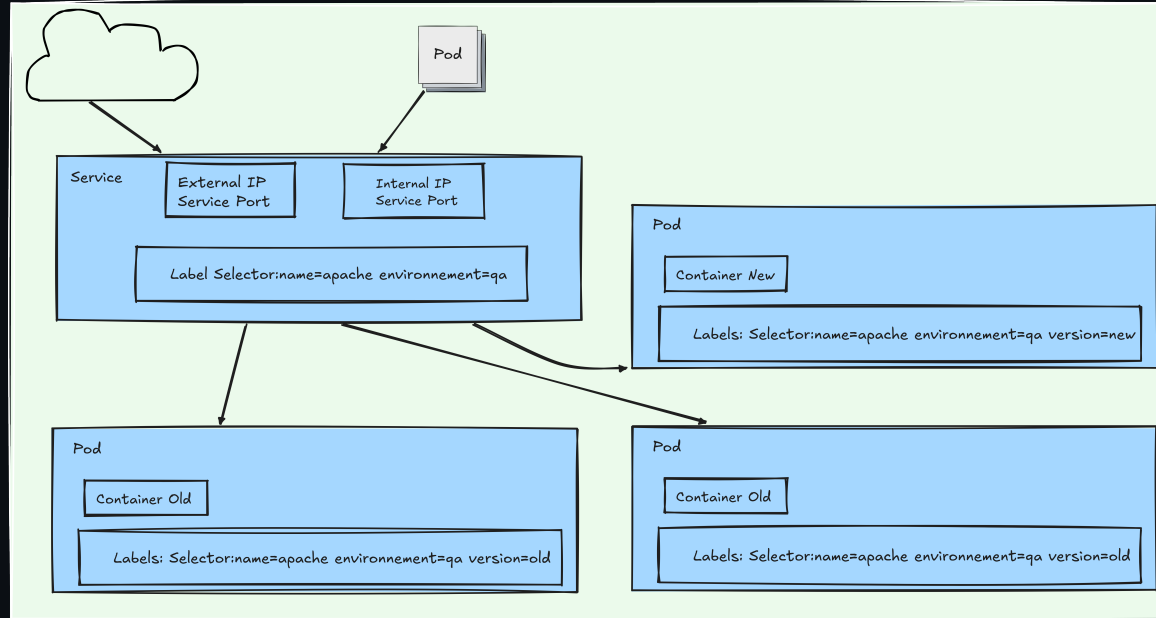
} (1) Sélection sur le label

KUBERNETES : LABELS

Organiser et sélectionner les objets Kubernetes

- Système de clé/valeur (list of key=value pairs), attaché à tous les objets
- Organiser les objets Kubernetes (PODs, RC, Services, etc.) de manière cohérente (structure de l'application)
- Corréler / sélectionner des éléments :
 - Matching pods to replication controllers
 - Matching pods to services
- Les objets peuvent être interrogés via l'API server par label

Services and Labels



KUBERNETES : LABELS

Exemple de label dans un manifeste

- Exemple de label :

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

} ⁽¹⁾ Marque le Pod avec le
label app=nginx

KUBERNETES : NAMESPACES

Partitionner le cluster en espaces logiques

- Fournissent une séparation logique des ressources (par utilisateurs, projets/applications, etc.)
- Attachés à chaque objet : un objet vit dans un namespace donné
 - Évite les collisions de noms
- Isolation fonctionnelle :
 - Pods in ns1 will not get service variable from ns2
 - Droits/permissions (RBAC) peuvent différer entre ns1 et ns2 (CRUD)
- Attention : **“le réseau n’est pas segregated !”**

KUBERNETES : NAMESPACES

Pourquoi le réseau n'est pas “segregated” par défaut

- Un namespace = isolation logique (noms, RBAC, quotas), pas une isolation réseau
- Par défaut, le réseau du cluster est “plat” :
 - Chaque Pod a une IP routable dans tout le cluster (via le CNI)
 - Le routage ne tient pas compte des namespaces
- La séparation DNS n'empêche pas l'accès :
 - On peut joindre un Service d'un autre namespace via son FQDN
(svc.ns2.svc.cluster.local)
- Pour segmenter réellement, il faut des règles réseau :
 - NetworkPolicy + un CNI qui les applique (ex: Calico, Cilium)

KUBERNETES : VOLUMES

Rendre les données persistantes pour les pods

- Fournir du stockage persistant aux PODs
- Fonctionnent de la même façon que les volumes Docker pour
 - les volumes hôte :
 - EmptyDir = volumes docker
 - HostPath = volumes hôte
- Support de multiples backend de stockage :
 - GCE : PD
 - AWS : EBS
 - GlusterFS / NFS
 - Ceph
 - iSCSI

KUBERNETES : VOLUMES

Monter un volume dans un Pod

- On déclare d'abord le volume et on l'affecte au Pod :
- Exemple :

```
apiVersion: v1
kind: Pod
metadata:
  name: redis
spec:
  containers:
    - name: redis
      image: redis
      volumeMounts:
        - name: redis-persistent-storage
          mountPath: /data/redis
```

```
volumes:
  - name: redis-persistent-storage
    emptyDir: {}
```

KUBERNETES : NETWORKING

Réseau inter-nœuds sans NAT via CNI

- Les conteneurs peuvent communiquer sans NAT
 - Un nœud peut accéder aux conteneurs des autres nœuds sans NAT
- Nécessite une solution tierce :
 - Canal : Flannel + Calico
 - Weaves
 - OpenShift
 - OpenContrail
 - Romana
- Ces solutions implémentent CNI (Container Network Interface)
 - Spécifications sur la configuration d'interface réseaux des conteneurs

Networking Setup

Flannel : un CNI simple pour démarrer

- Flannel
 - Super super easy configuration
 - Can create a vxlan overlay network
 - Can configure docker to launch pods in this overlay
 - Pods just work!
- There are many other solutions.
 - This one is easy.

KUBERNETES : NETWORKPOLICY

Définir des règles de filtrage entre pods

- Contrôle la communication entre les PODs au sein d'un namespace
- Pare-feu entre les éléments composant une application
- Exemple (simplifié) :

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: test-network-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  ingress:
    - from:
```

```
- namespaceSelector:
    matchLabels:
      project: myproject
- podSelector:
    matchLabels:
      role: frontend
egress: []
policyTypes:
- Ingress
- Egress
```

KUBERNETES : DEPLOYMENTS

Gérer les mises à jour et le cycle de vie des pods

- Permet d'assurer le fonctionnement d'un ensemble de PODs
 - Version, Update et Rollback
- Souvent combiné avec un objet de type service

KUBERNETES : INGRESS RESOURCE

Routage HTTP/HTTPS vers les services

- Définition de règles de routage applicatives (HTTP/HTTPS)
 - Traffic load balancing, SSL termination, name based virtual hosting
- Définies dans l'API et ensuite implémentées par un Ingress Controller
- Exemple :

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  namespace: default
  name: traefik
spec:
  ingressClassName: traefik
  rules:
  - host: traefik.archifleks.net
```

```
http:
  paths:
  - path: /
    pathType: Prefix
    backend:
      service:
        name: traefik-console
        port:
          number: 8080
```

KUBERNETES : INGRESS RESOURCE

Exemple de manifeste Ingress

- Exemple :

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  namespace: default
  name: traefik
spec:
  ingressClassName: traefik
  rules:
  - host: traefik.archifleks.net
    http:
      paths:
      - path: /
        pathType: Prefix
```

yml

```
backend:
  service:
    name: traefik-console
  port:
    number: 8080
```

KUBERNETES : INGRESS CONTROLLER

Le composant qui applique les règles Ingress

- Routeur applicatif de bordure (L7 Load Balancer)
- Implémente les Ingress Resources
- Plusieurs implémentations :
 - Traefik (<https://traefik.io/>)
 - nginx

KUBERNETES : RUN ET ADMINISTRATION

Outils et objets pour exploiter le cluster

- WebUI (Kubernetes Dashboard)
- Kubectl (Outil CLI)
- Objets: Secret et ConfigMap : paramétrages, plus sécurisés que les variables d'environnements
 - Vault et consul

KUBERNETES : AUJOURD'HUI

État de maturité et adoption

- Version 1.35.0 : stable en production
- Solution complète et une des plus utilisées
- Éprouvée par Google

KUBERNETES : ARCHITECTURE

KUBERNETES : COMPOSANTS

Une architecture basée sur des binaires autonomes

- Kubernetes est écrit en Go, compilé statiquement.
- Un ensemble de binaires sans dépendances
- Faciles à conteneuriser et à packager
- Peut se déployer uniquement avec des conteneurs sans dépendances d'OS

KUBERNETES : COMPOSANTS

Rôles des principaux services Kubernetes

- kube-apiserver : API server qui permet la configuration d'objet Kubernetes (Pods, Service, Replication Controller, etc.)
- kube-proxy : Permet le forwarding TCP/UDP et le load balancing entre les services et les backend (Pods)
- kube-scheduler : Implémente les fonctionnalités de scheduling
- kube-controller-manager : Responsable de l'état du cluster, boucle infinie qui régule l'état du cluster afin d'atteindre un état désiré

KUBERNETES : COMPOSANTS

Agents et outils côté nœuds

- kubelet : Service “agent” fonctionnant sur tous les nœuds et assure le fonctionnement des autres services
- kubectl : Client qui permet de piloter un cluster Kubernetes

KUBERNETES : KUBELET

Agent de nœud qui applique l'état désiré

- Service principal de Kubernetes
- Permet à Kubernetes de s'auto configurer :
 - Surveille un dossier contenant les manifests (fichiers YAML des différents composant de K8s).
 - Applique les modifications si besoin (upgrade, rollback).
 - Surveille l'état des services du cluster via l'API server (kube-apiserver).
- Dossier de manifest sur un noeud master :
 - /etc/kubernetes/manifests/

KUBERNETES : KUBELET

Exemple de manifest supervisé par kubelet

- Exemple du manifest kube-proxy :

```
apiVersion: v1
kind: Pod
metadata:
  name: kube-proxy
  namespace: kube-system
spec:
  hostNetwork: true
  containers:
  - name: kube-proxy
    image: quay.io/coreos/hyperkube:v1.3.6_coreos.0
    command:
    - /hyperkube
    - proxy
```

YAML

```
- --master=http://127.0.0.1:8080
- --proxy-mode=iptables
```

KUBERNETES : KUBE-APISERVER

Point central de contrôle via l'API REST

- Les configurations d'objets (Pods, Service, RC, etc.) se font via l'API server
- Un point d'accès à l'état du cluster aux autres composants via une API REST
- Tous les composants sont reliés à l'API server

KUBERNETES : KUBE-SCHEDULER

Décider où placer les pods

- Planifie les ressources sur le cluster
- En fonction de règles implicites (CPU, RAM, stockage disponible, etc.)
- En fonction de règles explicites (règles d'affinité et anti-affinité, labels, etc.)

KUBERNETES : KUBE-PROXY

Routage et load-balancing des Services

- Responsable de la publication de services
- Utilise iptables
- Route les paquets à destination des PODs et réalise le load balancing TCP/UDP

KUBERNETES : KUBE-CONTROLLER-MANAGER

Contrôleurs qui maintiennent l'état désiré

- Boucle infinie qui contrôle l'état d'un cluster
- Effectue des opérations pour atteindre un état donné
- De base dans Kubernetes : replication controller, endpoints controller, namespace controller et serviceaccounts controller

KUBERNETES : NETWORK-POLICY-CONTROLLER

Appliquer les règles NetworkPolicy via le CNI

- Implémente l'objet NetworkPolicy
- Contrôle la communication entre les PODs
- Externe à Kubernetes et implémenté par la solution de Networking choisie :
 - OpenShift
 - OpenContrail
 - Romana
 - Calico

KUBERNETES : CONCLUSION

Notions clés : Node et Pod

- Node : Un node (“noeud” en fr) est une machine de travail du cluster Kubernetes. Ce sont des unités de travail qui peuvent être physiques, virtuelles mais aussi des instances cloud.
- Pod : Il s’agit de l’unité la plus petite de K8s, un pod encapsule le ou les conteneur(s) formant votre application conteneurisée partageant ainsi la même stack réseau (chaque pod se voit attribuer une adresse IP unique) et le même stockage, plus précisément un volume partagé (tous les conteneurs

du pod peuvent accéder aux volumes partagés, ce qui permet à ces conteneurs de partager plus facilement des données).

KUBERNETES : CONCLUSION

Objets clés : scaling, déploiement et exposition réseau

- Replicas : c'est le nombre d'instances d'un Pod ("réplique" en fr)
- ReplicaSet : s'assure que les réplicas spécifiés sont actifs
- Deployment : définit l'état désiré et fournit des mises à jour déclaratives de vos Pods et ReplicaSets.
- Service : Un service peut être défini comme un ensemble logique de pods exposés en tant que service réseau. C'est un niveau d'abstraction au-dessus du pod, qui fournit une adresse IP et un nom DNS unique pour un ensemble de pods. Avec les Services, il est très facile de gérer la configuration de Load Balancing

KUBERNETES : CONCLUSION